

AI Movie Recommendation System Documentation

Submitted By

- Malak AL-Sinani
- Aliya AL-Isaii
- Anoud AL-Saidi
- Sheika AL-Busafi

Course Information

Object-Oriented Programming and Artificial Intelligence Concepts

Project Type

Console-Based AI Application Using C# and .NET

Project Title

AI-Powered Movie Recommendation System Using C# Console Application

1. Project Overview

The AI Movie Recommendation System is a console-based application developed using C# and Object-Oriented Programming (OOP) concepts. The system allows users to register, log in, browse movies, search for movies, rate movies, and receive personalized movie recommendations.

The project simulates the functionality of modern streaming platforms by using Artificial Intelligence recommendation techniques such as Content-Based Filtering and Collaborative Filtering.

The system stores data using JSON files and provides an interactive user experience through a professional console interface.

2. Objectives of the Project

The main objectives of this project are:

- Build a movie recommendation system using C#.
- Apply Object-Oriented Programming concepts.
- Implement Artificial Intelligence recommendation logic.
- Store and manage data using JSON files.
- Create a professional and user-friendly console application.
- Apply LINQ queries for searching and filtering data.
- Improve user experience using menus, colors, and animations.

3. Features of the System

User Features

- Register new account
- Login to the system
- Logout from the system
- Browse movies
- Search movies
- View movie details
- Rate movies
- Update ratings
- Remove ratings

- View watch history
- View favorite movies
- View trending movies
- View top rated movies
- View system statistics
- Receive AI movie recommendations

4. Technologies Used

Technology	Purpose
C#	Main programming language
.NET	Application framework
JSON	Data storage
LINQ	Searching and filtering
Visual Studio	Development environment
Git & GitHub	Version control

5. Object-Oriented Programming Concepts

The project applies several OOP concepts.

5.1 Encapsulation

Encapsulation is used by keeping data inside classes and controlling access through properties and methods.

Example:

```
public class Movie
{
    public int Id { get; set; }
    public string Title { get; set; }
}
```

5.2 Abstraction

Abstraction is applied using interfaces.

Example:

```
public interface IRecommendation
{
    List<Movie> Recommend(User user, List<Movie> movies,
        List<Rating> ratings, List<User> users);
}
```

5.3 Polymorphism

Polymorphism is used when multiple recommendation strategies implement the same interface.

Example:

```
new ContentBasedService(),
new CollaborativeFilteringService()
```

Both classes use the same interface but provide different recommendation logic.

5.4 Inheritance

Inheritance can be applied in future improvements for extending user roles or movie categories.

6. Artificial Intelligence Logic

The system uses recommendation algorithms to provide personalized movie suggestions.

6.1 Content-Based Filtering

This technique recommends movies based on:

- Favorite genres
- Movie tags
- Previously watched movies
- User preferences

Example:

If a user likes Sci-Fi movies, the system recommends similar Sci-Fi movies.

6.2 Collaborative Filtering

This technique compares users with similar movie tastes.

If two users rate movies similarly, the system recommends movies liked by one user to the other user.

6.3 Cosine Similarity

Cosine Similarity is used to calculate similarity between users based on their ratings.

Formula:

$$\text{Similarity}(A,B) = (A \cdot B) / (||A|| \cdot ||B||)$$

The closer the result is to 1, the more similar the users are.

6.4 Weighted Recommendation Scoring

The recommendation score is calculated using:

- Genre matching
- User similarity
- Average movie ratings
- Movie popularity
- Tags

This creates smarter and more accurate recommendations.

7. System Architecture

The system is divided into multiple layers and classes.

Main Components

Models

- Movie
- User
- Rating

Services

- MovieService
- SearchService
- RecommendationService
- ContentBasedService
- CollaborativeFilteringService
- AuthenticationService

Utilities

- ValidationHelper
- ConsoleUI
- FileManager

Interfaces

- IRecommendation

8. JSON Data Storage

The project uses JSON files to store data permanently.

Files Used

File	Purpose
movies.json	Stores movies
users.json	Stores users
ratings.json	Stores ratings

9. LINQ Usage

LINQ is used for:

- Searching movies
- Filtering recommendations
- Sorting ratings
- Grouping data
- Calculating averages
- Trending movie analysis

Example:

```
movies.Where(m => m.Genre == "Sci-Fi")
```

10. User Interface Design

The project includes a professional console UI.

UI Improvements

- Colored menus
- Animated loading messages
- Professional dashboard
- Tables for movie display
- Error message boxes
- Recommendation panels
- Statistics dashboard

11. Main Dashboard Features

Browse Movies

Displays all available movies.

Search Movies

Searches movies using keywords.

Rate Movies

Users can rate movies from 1 to 5.

Remove Rating

Users can remove previous ratings.

Watch History

Displays watched movies.

AI Recommendations

Displays personalized recommendations.

Top Rated Movies

Displays highest rated movies.

Trending Movies

Displays movies with the highest number of ratings.

System Statistics

Displays:

- Total movies
- Total users
- Total ratings
- Average rating
- Most popular genre

12. Validation and Error Handling

The system validates:

- Username input
- Password input
- Rating values
- Numeric inputs
- Movie IDs

Error messages are displayed using colored warning boxes.

13. Challenges Faced

Several challenges were faced during development.

Example Challenges

- Git merge conflicts
- JSON formatting issues
- File path problems
- Recommendation accuracy
- Handling duplicate ratings
- Console UI design
- Data synchronization

These problems were solved through debugging and testing.

14. Future Improvements

Possible future improvements include:

- GUI version using Windows Forms or WPF
- Admin dashboard
- Real database integration
- Online movie APIs

- Machine learning models
- User profile pictures
- Multi-language support
- Movie trailers and posters

15. Conclusion

The AI Movie Recommendation System successfully demonstrates the use of Object-Oriented Programming, Artificial Intelligence recommendation techniques, JSON data persistence, and professional console design.

The system provides users with an interactive and personalized movie recommendation experience while applying important software engineering concepts such as interfaces, polymorphism, abstraction, and LINQ.

The project also demonstrates teamwork, problem solving, and practical implementation of AI concepts in a real-world application.